



TIRAHOU

Manuel d'apprentissage — Langages et frameworks de TIRAHOU

Comprendre les langages et frameworks derrière TIRAHOU, illustrés
par des extraits réels du code du projet.

Document généré automatiquement à partir de
docs/MANUEL_APPRENTISSAGE_TECHNOLOGIES.md

Ce manuel n'est pas un tutoriel généraliste : chaque concept est expliqué puis immédiatement illustré par un **extrait réel du code de TIRAHOU** — l'objectif est de comprendre *pourquoi* chaque technologie a été choisie et *comment* elle est concrètement utilisée dans le projet, pas de réapprendre un langage depuis zéro.

Vue d'ensemble de la stack

Couche	Langage	Framework principal	Rôle
Backend	Python 3.13	Django 5.2 + Django REST Framework	API REST, base de données, logique métier
Frontend web	TypeScript	React 19 + Vite	Interface utilisateur (PWA)
Mobile	TypeScript	React Native (Expo 57)	App étudiant/enseignant iOS/Android
Base de données	SQL	PostgreSQL (SQLite en local)	Persistance des données
Tâches asynchrones	Python	Celery + Redis	Emails, notifications différées
Déploiement	—	Render (backend) + Vercel (frontend)	Hébergement, CI/CD automatique

Partie 1 — Backend

1.1 Python — le langage

Pourquoi Python ? Lisibilité (proche du pseudo-code), écosystème mature pour le web (Django) et l'IA (SDK Anthropic), typage optionnel (type hints) qui apporte de la sécurité sans sacrifier la souplesse.

Concepts utilisés dans TIRAHOU

Classes et héritage — chaque modèle de données hérite d'une base commune :

```
# apps/core/models.py
class BaseModel(UUIDModel, TimeStampedModel):
    is_active = models.BooleanField(default=True)
    class Meta:
        abstract = True
```

`Student`, `Grade`, `Invoice` ... héritent tous de `BaseModel` : ils obtiennent automatiquement un `id` (UUID), `created_at`, `updated_at`, `is_active` — sans dupliquer le code (principe DRY : *Don't Repeat Yourself*).

Décorateurs — modifient le comportement d'une fonction sans la réécrire. Omniprésents dans TIRAHOU pour déclarer des routes API :

```
# apps/documents/pdf_views.py
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def generate_certificat_pdf(request, student_id):
    ...
```

Type hints — annotations de type, vérifiées par des outils externes (non bloquantes à l'exécution, mais documentent l'intention) :

```
def predict_student_success(student_id) -> dict | None:
```

Gestionnaires de contexte (`with`) — garantissent qu'une ressource est proprement fermée même en cas d'erreur :

```
with connection.cursor() as cursor:
    cursor.execute("SELECT set_config('app.current_student_id', %s, true)", [student_id])
```

Compréhensions de liste — écriture concise de transformations de collections, très utilisées dans les serializers et calculs de résultats :

```
grades = [{'ec_name': g.ec.name, 'final_grade': float(g.final_grade)} for g in student_grades]
```

1.2 Django — le framework web

Philosophie : *batteries included* — ORM, système de migrations, interface d'administration, authentification et sécurité (CSRF, XSS, SQL injection) fournis nativement, pour se concentrer sur la logique métier.

L'ORM (Object-Relational Mapping)

Chaque classe Python = une table SQL, chaque attribut = une colonne. Aucun SQL à écrire à la main dans 95% des cas :

```
# apps/evaluation/models.py
class Grade(BaseModel):
    student = models.ForeignKey(Student, on_delete=models.CASCADE, related_name='grades')
    ec = models.ForeignKey(EC, on_delete=models.CASCADE)
    cc_grade = models.DecimalField(max_digits=4, decimal_places=2, null=True)
    exam_grade = models.DecimalField(max_digits=4, decimal_places=2, null=True)
    final_grade = models.DecimalField(max_digits=4, decimal_places=2, null=True)
```

Requête associée : `Grade.objects.filter(student=student, status='publiee')` génère un `SELECT ... WHERE student_id = ... AND status = 'publiee'` — sans risque d'injection SQL (les valeurs sont automatiquement échappées).

Migrations

Chaque changement de modèle génère un fichier versionné qui décrit comment faire évoluer le schéma SQL, en avant (`migrate`) ou en arrière :

```
python manage.py makemigrations # génère le fichier de migration
python manage.py migrate        # l'applique à la base
```

TIRAHOU compte plus de 30 migrations, dont une conditionnelle qui n'active la Row-Level Security PostgreSQL que si le moteur de base le supporte (`apps/core/migrations/0002_enable_row_level_security.py`).

Vues et routage

Django associe une URL à une fonction (ou classe) Python :

```
# config/urls.py
path(API_V1, include('apps.evaluation.urls')),
```

Middleware

Code exécuté sur *chaque* requête, avant ou après la vue — utilisé dans TIRAHOU pour envelopper chaque requête PostgreSQL dans une transaction (nécessaire à la Row-Level Security, voir §1.5) :

```
# apps/core/middleware.py
class PostgresRLSTransactionMiddleware:
    def __call__(self, request):
        if connection.vendor != 'postgresql':
            return self.get_response(request)
        with transaction.atomic():
            return self.get_response(request)
```

1.3 Django REST Framework (DRF)

Django gère des pages HTML par défaut ; DRF ajoute la couche nécessaire pour exposer une **API REST en JSON**, consommée par le frontend React et l'app mobile.

Serializers — convertir Python ↔ JSON

```
# apps/admissions/serializers.py
class ApplicationSerializer(serializers.ModelSerializer):
    status_display = serializers.CharField(source='get_status_display', read_only=True)
    class Meta:
        model = Application
        fields = '__all__'
        read_only_fields = ['application_number', 'submitted_at', 'applicant']
```

Un serializer valide aussi les données entrantes (types, champs obligatoires) avant toute écriture en base.

ViewSets — regrouper les opérations CRUD

Une seule classe fournit lister/créer/lire/modifier/supprimer :

```
class ApplicationViewSet(viewsets.ModelViewSet):
    queryset = Application.objects.all()
    serializer_class = ApplicationSerializer
    permission_classes = [permissions.IsAuthenticated]

    @action(detail=True, methods=['post'])
    def submit(self, request, pk=None):
        ...
```

`@action` ajoute une route personnalisée (`POST /applications/{id}/submit/`) en plus des routes CRUD standard générées automatiquement par le `router` .

Permissions

TIRAHOU définit son propre système de permissions fines (RBAC) au-dessus de celui de DRF :

```
# apps/accounts/permissions.py
class HasModulePermission(BasePermission):
    def has_permission(self, request, view):
        ...
        return user.has_permission(module, action)
```

1.4 Authentification JWT (JSON Web Token)

Concept : après connexion, le serveur délivre un **jeton signé** contenant l'identité de l'utilisateur — le client le renvoie dans l'en-tête `Authorization: Bearer <token>` à chaque requête, sans que le serveur ait besoin de garder une session en mémoire (architecture *stateless*).

TIRAHOU utilise deux jetons : un `access` token de courte durée (8h) et un `refresh` token (7 jours) pour en obtenir un nouveau sans se reconnecter. Le frontend intercepte automatiquement les réponses `401` pour rafraîchir le jeton de façon transparente (`frontend/src/lib/axios.ts`). Voir aussi `apps/accounts/authentication.py` : une sous-classe de l'authentification JWT standard, étendue pour positionner le contexte PostgreSQL RLS dès qu'un utilisateur est authentifié.

1.5 PostgreSQL et les concepts relationnels

- **Clé primaire / étrangère** : chaque `ForeignKey` Django devient une contrainte SQL réelle (`REFERENCES`), garantissant l'intégrité des données (impossible de créer une note pour un étudiant inexistant).
- **Row-Level Security (RLS)** : mécanisme PostgreSQL natif qui filtre les lignes visibles *au niveau de la base elle-même*, indépendamment du code applicatif — implémenté dans TIRAHOU en défense en profondeur sur les notes et factures (`apps/core/rls.py`), avec des *politiques* SQL : `sql CREATE POLICY grades_student_isolation ON grades USING ( current_setting('app.is_staff', true) = 'true' OR student_id::text = current_setting('app.current_student_id', true) );`

1.6 Celery et Redis — tâches asynchrones

Certaines opérations (envoi d'email, notification différée) ne doivent pas bloquer la réponse HTTP.

Redis sert de file d'attente (*broker*) ; **Celery** exécute les tâches en arrière-plan, dans un processus séparé du serveur web.

1.7 Autres bibliothèques backend notables

Bibliothèque	Rôle dans TIRAHOU
<code>reportlab</code>	Génération des PDF (relevés, factures, cartes étudiant)
<code>qrcode</code>	QR codes de vérification sur les documents générés
<code>pyotp</code>	Double authentification (TOTP)
<code>pywebpush</code>	Notifications push web (VAPID)
<code>anthropic</code>	SDK officiel Claude — assistant IA (chatbot)
<code>drf-spectacular</code>	Génère la documentation API interactive (/api/docs/)
<code>whitenoise</code> + <code>gunicorn</code>	Servir l'app en production sur Render

Partie 2 — Frontend web

2.1 TypeScript — le langage

Pourquoi TypeScript plutôt que JavaScript ? Ajoute un système de types statiques vérifié *avant* l'exécution — la majorité des bugs d'intégration frontend/backend sont détectés à l'écriture plutôt qu'en production.

Concepts utilisés dans TIRAHOU

Interfaces / types — décrivent la forme exacte des données échangées avec l'API (générés à partir des serializers Django) :

```
// frontend/src/types/index.ts
export interface ChatMessage {
  id: string
  role: 'user' | 'assistant'
  content: string
  tools_used: string[]
  created_at: string
}
```

Génériques — un type paramétrable par un autre type, utilisé pour la pagination (toutes les listes API renvoient la même structure, quel que soit le type d'objet) :

```
export interface PaginatedResponse<T> {  
  count: number  
  next: string | null  
  results: T[]  
}
```

Async/await — écrire du code asynchrone (requêtes réseau) de façon linéaire plutôt qu'en callbacks imbriqués :

```
const res = await api.get<PaginatedResponse<Student>>('/students/')
```

2.2 React 19

Concept central : l'interface est une fonction de l'état (`UI = f(state)`) — décrite via des **composants** réutilisables qui se redessinent automatiquement quand leurs données changent.

JSX — syntaxe qui mélange HTML et JavaScript/TypeScript dans le même fichier :

```
{unread > 0 && (  
  <span className="...">{unread > 9 ? '9+' : unread}</span>  
)}
```

Hooks — fonctions qui donnent accès à l'état et au cycle de vie d'un composant *sans* écrire de classe : - `useState` : état local d'un composant (ex. `const [open, setOpen] = useState(false)`) - `useEffect` : exécuter du code en réaction à un changement (ex. scroller vers le bas d'une conversation à chaque nouveau message) - Hooks personnalisés : `useRole()`, `useDebounce()` — encapsulent une logique réutilisable propre au projet.

2.3 Vite — l'outil de build

Remplace Webpack/Create React App : démarrage quasi instantané en développement (*Hot Module Replacement*), build de production optimisé. Génère aussi le Service Worker de la PWA (`vite-plugin-pwa`), qui permet l'installation de TIRAHOUE comme application et le fonctionnement partiel hors-ligne.

2.4 TailwindCSS

Approche *utility-first* : au lieu d'écrire du CSS séparé, on compose des classes utilitaires directement dans le JSX :

```
<div className="flex items-center gap-3 rounded-xl bg-white dark:bg-slate-900 px-3 py-2">
```

Chaque classe fait une seule chose (`flex` , `gap-3` , `rounded-xl` ...) — le mode sombre (`dark:`) est géré par un simple préfixe de classe, sans CSS dupliqué.

2.5 TanStack React Query — état serveur

Gère le cycle de vie complet des données venant de l'API : cache automatique, re-fetch en arrière-plan, invalidation après une écriture, état de chargement/erreur — sans avoir à gérer ça manuellement avec `useState` / `useEffect` :

```
const { data, isLoading } = useQuery({
  queryKey: ['chatbot-conversations'],
  queryFn: () => chatbotApi.getConversations().then(r => r.data.results),
})

const sendMessage = useMutation({
  mutationFn: ({ id, content }) => chatbotApi.sendMessage(id, content),
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['chatbot-conversations'] }),
})
```

2.6 Zustand — état global (client)

Contrairement à React Query (données *serveur*), Zustand gère l'état *purement client* partagé entre composants — utilisé dans TIRAHOU pour l'authentification (`authStore`) : utilisateur connecté, jetons JWT, persistant dans le `localStorage` . Choisi pour sa simplicité par rapport à Redux (pas de *reducers/actions* boilerplate).

2.7 React Router

Routage déclaratif côté client (*Single Page Application* : une seule page HTML, la navigation change uniquement le composant affiché, sans recharger le serveur) :

```
<Route path="/my-grades" element={<MyGradesPage />} />
```

Combiné à `RoleBasedRoute` (composant maison) pour restreindre certaines routes selon le rôle de l'utilisateur connecté.

2.8 Autres bibliothèques frontend notables

Bibliothèque	Rôle dans TIRAHOU
Radix UI / Headless UI	Composants accessibles sans style imposé (menus, dialogues, onglets)
React Hook Form + Zod	Formulaires performants + validation de schéma partagée
Recharts	Graphiques du tableau de bord analytics
Framer Motion	Animations de transition entre pages
Axios	Client HTTP (intercepteurs pour le rafraîchissement JWT automatique)
Playwright	Tests de bout en bout (E2E) du parcours utilisateur réel

Partie 3 — Application mobile

3.1 React Native & Expo

React Native permet d'écrire l'app mobile en React/TypeScript, compilée en composants natifs iOS/Android (pas de WebView) — le même langage et beaucoup de concepts que le frontend web (hooks, état, TypeScript), mais des composants différents (`View` / `Text` au lieu de `div` / `span`).

Expo simplifie le développement React Native : pas besoin d'installer Xcode/Android Studio pour itérer, prévisualisation sur téléphone via l'app Expo Go, et **Expo Router** apporte un routage *basé sur les fichiers* (chaque fichier dans `app/` devient automatiquement une route), comme Next.js.

Partie 4 — Infrastructure et déploiement

4.1 Git & GitHub

Système de contrôle de version : chaque modification est un *commit* horodaté et attribué, ce qui permet de revenir en arrière, de comprendre l'historique d'une fonctionnalité, et de collaborer sans écraser le travail des autres. TIRAHOU suit une convention de messages de commit préfixés (`feat:` , `fix:` , `docs:` , `chore:`).

4.2 Render & Vercel — hébergement et CI/CD

- **Render** héberge le backend Django (+ PostgreSQL managé) : chaque `git push` sur `main` déclenche automatiquement `backend/build.sh` (installation des dépendances, migrations, collecte des fichiers statiques, seed des comptes de démonstration).
- **Vercel** héberge le frontend (build Vite statique) : même principe, déploiement automatique à chaque push.

Aucune intervention manuelle sur un serveur : c'est le principe de *Continuous Deployment*.

4.3 Docker

Utilisé ponctuellement dans le projet (pas en production) pour créer un environnement PostgreSQL isolé et reproductible, notamment pour valider réellement la Row-Level Security avant de l'activer en production (voir §1.5) — sans installer PostgreSQL directement sur la machine.

Glossaire récapitulatif

Terme	Définition courte
ORM	Object-Relational Mapping — manipuler une base SQL avec des objets du langage
API REST	Interface où chaque ressource (étudiant, note...) a une URL, manipulée via HTTP (GET/POST/PATCH/DELETE)
JWT	Jeton signé prouvant l'identité d'un utilisateur, sans état serveur
Migration	Fichier versionné décrivant un changement de schéma de base de données
Middleware	Code exécuté sur chaque requête HTTP, avant/après la vue
Serializer	Convertit un objet Python en JSON (et valide le sens inverse)
Hook (React)	Fonction donnant accès à l'état/cycle de vie d'un composant fonctionnel
State management	Gestion des données partagées entre composants d'une interface
RLS	Row-Level Security — filtrage des lignes visibles au niveau de la base de données
CI/CD	Intégration/déploiement continu — automatisation du build et de la mise en ligne
PWA	Progressive Web App — site web installable et partiellement utilisable hors-ligne